

LLM Gateway

with Rate Limiting, Fallback Routing & Observability

Product Requirements Document — Version 2.2

Project #11 — AI Engineering Portfolio

1. Project Overview

This PRD defines a production-style LLM API Gateway — an infrastructure layer that manages all LLM API traffic across multiple teams. It enforces per-team rate limits (RPM and TPM) via atomic dual-bucket admission, reserves and reconciles token budgets per provider attempt, automatically fails-over to alternate providers, and provides unified observability across every model call.

This is a personal portfolio project. The objective is not to simulate enterprise production scale. The objective is to build a technically credible, deeply understood, locally demonstrable system that generates strong backend/AI infrastructure interview talking points and can appear on a CV with real, measured benchmark numbers.

1.1 Why This Project Lands Interviews

Every company with more than one team using LLMs ends up building exactly this. It is pure infrastructure engineering applied to AI — precisely the skill-set that AI teams desperately need. This project lets you lead with production engineering strengths while demonstrating deep AI-systems knowledge.

1.2 Core Design Principles

- Finish-ability over completeness — core features first, stretch goals clearly labelled.
- Zero paid API calls in any automated test — MockLLMProvider covers all CI.
- GitHub Copilot Pro workflow — all implementation prompts are small, incremental, and sequentially testable.
- Measured results only — no benchmark numbers are invented; README contains actual load-test output.

2. Goals & Success Metrics

2.1 Primary Goals

- Ship a production-hardened LLM gateway with multi-provider routing, fallback, and budget enforcement.
- Demonstrate full-stack observability: OpenTelemetry → Jaeger, Prometheus → Grafana, structured JSON logs.
- Produce a portfolio artefact with quantified headline numbers: latency overhead, cost accuracy, throughput.
- All reliability features (retry, fallback, circuit breaker) are developable and testable without spending money.

2.2 Success Metrics

■ P99 gateway overhead < 10 ms is a TARGET, not a pre-committed guarantee. README/CV must contain

ACTUAL measured numbers.

Metric Target How Measured Gateway overhead latency P50 < 3 ms load test: measured provider duration subtracted Gateway overhead latency P99 < 10 ms (target) load test: measured provider duration subtracted Rate-limit enforcement 100% correct under Redis integration tests + load test accuracy concurrent load Fallback activation time < 2 s on primary integration test with MockLLMProvider provider outage Load test throughput ≥ 5,000 requests Locust report processed end-to-end Budget overruns in tests Zero — 100% cap integration test suite enforcement Grafana dashboards 3 dashboards: Ops, docker-compose up verification Business, Performance

3. Architecture & Tech Stack

All components are included from the start of the project. Datastores are assigned clear, non-overlapping responsibilities.

3.1 Technology Stack

Component Tool / Library Rationale Language Python 3.11+ Async-native; rich LLM ecosystem API Framework FastAPI High-throughput async; dependency injection; OpenAPI docs Rate Limiting / State Redis 7+ Distributed atomic state; Lua script support; sub-ms ops Durable Storage PostgreSQL 15+ Team metadata, API key metadata, audit log, durable records Observability — Traces OpenTelemetry SDK → OTLP Industry-standard distributed tracing Collector → Jaeger Observability — Metrics Prometheus + Grafana Operational and business metrics dashboards Observability — Logs Structured JSON (structlog) Machine-parseable; safe field filtering LLM Providers Groq / Google Gemini / OpenRouter / MockLLMProvider three independent cloud providers; MockLLMProvider for deterministic testing Containerisation Docker + docker-compose Full-stack local orchestration Pricing Config config/pricing.yaml (Decimal Independently updatable; never hardcoded values) in source Testing pytest-asyncio, httpx, real No fakeredis for Lua/concurrency; no paid Redis container API calls in CI Load Testing Locust Scriptable; custom stats reporter CI GitHub Actions Ruff, mypy, unit + Redis integration tests, Docker build

3.2 Datastore Responsibilities

Redis (ephemeral / coordination):

- Rate-limit token-bucket state (capacity, tokens, refill timestamp) — dual-bucket RPM + TPM
- Budget reservation counters (daily and monthly microdollar integers)
- Circuit-breaker state (shared across replicas)
- Passive provider health signals (EWMA latency, rolling error rate)
- Short-lived caches and ephemeral coordination

PostgreSQL (durable / auditable):

- Team metadata and configuration
- API key metadata (HMAC digests, prefix, permissions, revocation status)
- Admin audit log (immutable append-only records of all admin actions)
- Historical cost and usage records for long-term reporting

■ PostgreSQL is part of the architecture from Phase 0. The minimum schema (teams, api_keys) MUST exist

before API-key authentication is implemented. Do not introduce PostgreSQL as an afterthought.

3.3 High-Level Data Flow

Client request → FastAPI endpoint → Auth middleware (API key validation via HMAC-SHA-256, team lookup) → RoutingEngine (logical model tier → provider selection) → Rate limiter (atomic dual-bucket Lua: RPM + TPM) → Budget admission (pre-request reservation in Redis) → FallbackExecutor (retry + circuit breaker + provider fallback)
Each provider call = one ProviderAttempt(attempt_id, request_id, ...) → Provider adapter (Groq / Google Gemini / OpenRouter / Mock) → Budget reconciliation (actual cost per attempt → update Redis counters) → Response returned to client → OTel span closed; Prometheus metrics updated; structured log emitted

3.4 Observability Architecture

■ These three pillars are distinct systems. Prometheus does NOT store traces.

Metrics: Prometheus → Grafana Prometheus scrapes the gateway every 15 s. Grafana renders three dashboards: Operations, Business, Performance.

Traces: OpenTelemetry SDK → OTLP Collector → Jaeger Every request generates a root span with child spans for auth, rate-limit check, provider selection, each LLM call attempt, and budget reconciliation. request_id and attempt_id are correlated with trace_id in all logs. Jaeger is the explicit trace backend (Jaeger all-in-one in docker-compose).

Logs: Structured JSON (structlog) Every log line is valid JSON with a fixed set of safe fields. Prompts, completions, API keys, the secret pepper, and Authorization headers are NEVER logged.

4. Domain Models

All domain models are defined once in gateway/domain/ and used across all subsystems. Provider-SDK-specific types stay inside provider adapters and never leak into business logic.

4.1 Core Request/Response Models

ChatMessage(role, content) ChatCompletionRequest(messages, model_tier, max_tokens, temperature, stream, team_id, request_id) ChatCompletionResponse(text, input_tokens, output_tokens, latency_ms, model_id, provider, finish_reason, request_id) Usage(input_tokens, output_tokens, cost_microdollars) StreamingChunk(delta, finish_reason, index) ProviderMetadata(provider_name, model_id, raw_model_id) ModelConfig(provider, model_id, quality_tier, max_tokens, cost_per_input_token_microdollars, cost_per_output_token_microdollars)

4.2 Provider Attempt Models (V2.1 Addition)

A single gateway request may generate multiple provider attempts (retries and fallbacks). The following models make this explicit throughout the system.

ProviderAttempt(attempt_id: str, # UUID v4, generated per provider call request_id: str, # top-level correlation identifier (ties to gateway request) attempt_number: int, # 1-based; increments across retries and fallbacks provider: str, model: str, estimated_cost_microdollars: int, actual_cost_microdollars: int | None, # None until reconciled outcome: AttemptOutcome, # SUCCESS | RETRYABLE_FAILURE | NON_RETRYABLE_FAILURE | STREAMING_PARTIAL) AttemptOutcome(Enum): SUCCESS RETRYABLE_FAILURE NON_RETRYABLE_FAILURE STREAMING_PARTIAL request_id is the top-level correlation identifier for logs, traces, and API responses. attempt_id identifies individual provider executions. All structured log fields and OTel span attributes carry both identifiers.

4.3 Normalized Provider Exceptions

ProviderError(message, provider, model, status_code) # base ProviderTimeoutError(...) # retryable ProviderRateLimitError(..., retry_after_seconds) # retryable; honor Retry-After ProviderUnavailableError(..., status_code) # retryable (5xx) ProviderAuthenticationError(...) # non-retryable InvalidRequestError(...) # non-retryable InvalidProviderResponseError(...) # non-retryable

5. Provider Abstraction

Provider business logic must NOT be placed inside a single `send_llm_request()` function containing provider-switching conditionals. Instead, implement a clean interface with one concrete adapter per provider.

5.1 LLMProvider Interface

```
class LLMProvider(ABC):  
    async def complete(request: ChatCompletionRequest) -> ChatCompletionResponse  
    async def stream(request: ChatCompletionRequest) -> AsyncIterator[StreamingChunk]  
    async def health_check() -> ProviderHealth
```

5.2 Concrete Providers

- `GroqProvider` — wraps Groq SDK; translates Groq-specific exceptions into normalized gateway exceptions
- `GeminiProvider` — wraps Google Gemini SDK; translates Gemini response formats; supports streaming
- `OpenRouterProvider` — wraps OpenRouter REST API; supports OpenAI-compatible request format; translates OpenRouter errors
- `MockLLMProvider` — deterministic simulation; required from Phase 1 (see Section 6)

5.3 ProviderRegistry

A `ProviderRegistry` maps `(provider_name, model_id)` → `LLMProvider` instance. The gateway never instantiates provider adapters directly inside routing or endpoint code.

6. MockLLMProvider

■ MockLLMProvider MUST be implemented immediately after the provider interface — before any real provider. No automated test should require a paid API call.

6.1 Required Simulation Modes

Mode Configuration Used To Test Normal completion configurable text, latency_ms Happy-path routing, accounting, streaming Configurable latency latency_ms per mode EWMA health signals, P99 overhead measurement HTTP 429 fail_rate or always ProviderRateLimitError, Retry-After, backoff HTTP status_code param Circuit breaker, fallback routing 500/502/503/504 Timeout timeout_after_ms ProviderTimeoutError, retry, stream termination Connection failure connection_fail=True Network-level error handling Malformed response corrupt=True InvalidProviderResponseError handling Complete outage mode=OUTAGE All-providers-unavailable path, 503 Recovery mode=RECOVERY after N calls Circuit breaker HALF_OPEN → CLOSED

7. Rate Limiting

7.1 Algorithm: Distributed Token Bucket

■ The V1 PRD described Redis sorted-set sliding windows while calling the algorithm "token bucket". These are different algorithms. V2.1 uses an actual token bucket.

A token bucket maintains conceptual state of four values: capacity (maximum tokens), available tokens, refill rate (tokens per second), and last_refill_timestamp. Tokens are consumed on each request; the bucket refills continuously at the configured rate up to capacity.

7.2 Why Token Bucket

- Burst capacity: a team can consume its RPM allowance in a short burst rather than being hard-limited to one request per 1/RPM seconds. This matches real user behavior better than leaky bucket.
- Predictable retry: remaining capacity is deterministic and can be communicated via Retry-After headers.
- Redis required: process-local state would be incorrect under multiple gateway replicas. Redis holds the single authoritative bucket state.
- Lua atomicity: read + modify + write of bucket state must be a single atomic operation. Without Lua, a concurrent request on another replica could read stale available tokens between the read and the write, allowing over-consumption.

7.3 Two Independent Limits — Atomic Dual-Bucket Admission

■ RPM and TPM admission MUST be atomic. Separate Lua operations for RPM and TPM can partially consume capacity: RPM succeeds, TPM fails, but RPM capacity is already gone. This is incorrect.

Each team enforces two conceptually independent token buckets:

- RPM bucket — capacity = limit_rpm, refill rate = limit_rpm / 60 tokens per second
- TPM bucket — capacity = limit_tpm, refill rate = limit_tpm / 60 tokens per second Both limits are evaluated and consumed atomically inside a SINGLE Redis Lua operation. The single-bucket implementation may still exist for isolated unit testing, but all actual request admission MUST use the atomic dual-bucket Lua script described in Section 7.5.

7.4 Atomic Dual-Bucket Lua Script — Specification

The admission Lua script receives the following arguments: KEYS[1] = gateway:rl:{team_id}:rpm KEYS[2] = gateway:rl:{team_id}:tpm ARGV[1] = rpm_capacity ARGV[2] = rpm_refill_rate_per_second ARGV[3] = tpm_capacity ARGV[4] = tpm_refill_rate_per_second

```

ARGV[5] = tpm_tokens_requested (estimated_input_tokens + max_output_tokens)
ARGV[6] = current_timestamp_seconds (float)

```

The script MUST:

- 1. Load RPM bucket state from KEYS[1].
 - 2. Load TPM bucket state from KEYS[2].
 - 3. Refill both buckets based on elapsed time since last_refill_ts.
 - 4. Check whether RPM bucket has ≥ 1 token AND TPM bucket has $\geq$ tpm_tokens_requested.
 - 5a. If EITHER bucket lacks capacity: consume NOTHING from either bucket. Return [0, retry_after_ms] where retry_after_ms is the maximum wait time across both buckets.
 - 5b. If BOTH have sufficient capacity: consume 1 RPM token; consume tpm_tokens_requested TPM tokens; persist both updated bucket states atomically; return [1, remaining_rpm, remaining_tpm].
- The script returns a table: [allowed, remaining_rpm_or_retry_ms, remaining_tpm_or_nil]. -- Pseudocode (implement as actual Lua):
- ```

local now = tonumber(ARGV[6]) -- Load + refill RPM bucket
local rpm = load_bucket(KEYS[1])
rpm.available = math.min(rpm.capacity, rpm.available + rpm.refill_rate * (now - rpm.last_refill_ts))
rpm.last_refill_ts = now
-- Load + refill TPM bucket
local tpm = load_bucket(KEYS[2])
tpm.available = math.min(tpm.capacity, tpm.available + tpm.refill_rate * (now - tpm.last_refill_ts))
tpm.last_refill_ts = now
local rpm_ok = rpm.available >= 1
local tpm_ok = tpm.available >= tonumber(ARGV[5])
if not rpm_ok or not tpm_ok then -- consume NOTHING; calculate retry_after
local rpm_wait = rpm_ok and 0 or (1 - rpm.available) / rpm.refill_rate * 1000
local tpm_wait = tpm_ok and 0 or (tonumber(ARGV[5]) - tpm.available) / tpm.refill_rate * 1000
return {0, math.ceil(math.max(rpm_wait, tpm_wait))}
end
-- Both ok — consume and persist atomically
rpm.available = rpm.available - 1
tpm.available = tpm.available - tonumber(ARGV[5])
save_bucket(KEYS[1], rpm)
save_bucket(KEYS[2], tpm)
return {1, rpm.available, tpm.available}

```

## 7.5 Retry-After Calculation

When a request is rejected, Retry-After is the time in milliseconds until the limiting bucket will have enough tokens:  $(\text{tokens\_needed} - \text{available\_tokens}) / \text{refill\_rate\_per\_second} \times 1000$ , rounded up. The max of RPM-wait and TPM-wait is returned.

## 7.6 Redis Key Schema

gateway:rl:{team\_id}:rpm → hash: {capacity, available, refill\_rate, last\_refill\_ts}  
gateway:rl:{team\_id}:tpm → hash: {capacity, available, refill\_rate, last\_refill\_ts}

## 7.7 Acceptance Criteria

- Concurrent load at 2× RPM cap: zero requests slip through over the limit.
- Lua scripts verified with real Redis container (not fakeredis) for correct concurrency semantics.
- Retry-After header present on every 429 response.
- Test: a TPM rejection does NOT consume any RPM capacity.
- Test: an RPM rejection does NOT consume any TPM capacity.
- Test: both limits pass → exactly 1 RPM token and N TPM tokens consumed.
- Test: conservative TPM behaviour — after a request completes with `actual_output < max_output_tokens`, the TPM bucket is still charged `estimated_tpm_tokens (input + max_output)`, NOT `actual_tpm_tokens (input + actual_output)`. No TPM capacity is returned post-execution.

## 8. Cost Representation & Budget Enforcement

### 8.1 Numeric Representation

■ Binary floating-point (Python float) MUST NOT be used for authoritative cost accounting. Floating-point

arithmetic on currency values produces rounding errors that compound over many requests.

The system uses two representations at appropriate layers:

- Decimal — used at configuration boundaries (pricing.yaml) and domain model boundaries for human- readable values.
- Integer microdollars — used inside Redis for all atomic budget accounting. 1 USD = 1,000,000 microdollars. Integer arithmetic in Redis Lua is exact.

### 8.2 Pricing Configuration

Provider pricing is configuration-driven. It must be independently updatable without code changes.

```
config/pricing.yaml
EXAMPLE / PLACEHOLDER values – verify current pricing with each provider
```

```
providers: groq: llama-3.3-70b-versatile: cost_per_input_token: "0.000005" # Decimal string — example only
cost_per_output_token: "0.000015" # Decimal string — example only gemini: gemini-2.5-flash: cost_per_input_token:
"0.00000015" # example only cost_per_output_token: "0.00000060" # example only openrouter: openrouter-free:
cost_per_input_token: "0.00000015" # example only cost_per_output_token: "0.00000060" # example only
```

■ Example values above are illustrative. Do NOT hardcode real current pricing in source code. Copilot/Claude

must NOT be instructed to invent or hardcode "real 2025/2026 pricing".

### 8.3 TPM Rate-Limit Reservation Semantics (Separate from Financial Budget)

■ TPM rate-limit accounting and financial budget accounting are two separate systems. Do not conflate

them.

TPM rate-limit reservation — conservative, no post-request refund: Before provider execution, the atomic dual-bucket Lua script consumes the following amount from the TPM token bucket: `estimated_tpm_tokens = estimated_input_tokens + max_output_tokens` This TPM consumption is FINAL. After the provider attempt completes, the unconsumed portion is NOT returned to the TPM bucket. If actual output is 500 tokens but `max_output_tokens` was 4000, the TPM bucket is still charged 5000 tokens (`input + max_output`), not 1500 tokens (`input + actual_output`).

This is an intentional engineering trade-off:

- Guarantees safe distributed TPM enforcement with no race conditions.
- Keeps the rate limiter atomic and simple — one Lua operation, no post-call write-back.

- Avoids a second distributed reconciliation/refund operation on the rate-limit bucket.
- Sacrifices some TPM utilisation efficiency: teams with requests that regularly generate far fewer tokens than `max_output_tokens` will see lower effective TPM throughput than their limit.

■ Tests must NOT expect TPM bucket capacity to be restored after a request completes. The TPM bucket is

charged `estimated_tpm_tokens` at admission and that is the final charge for rate-limiting purposes.

Financial budget reservation — reconciled against actual cost: Financial budget accounting is entirely separate from TPM rate-limit accounting. The BudgetManager operates on microdollar integers and DOES reconcile after each provider attempt. See Section 8.4 for the full financial reservation/reconciliation flow.

The two systems use the same `estimated_tpm_tokens` figure as input to cost estimation, but diverge after that: TPM rate limiter: charges `estimated_tpm_tokens`, no refund after execution BudgetManager: reserves `estimated_cost_microdollars`, reconciles to `actual_cost_microdollars`

8.4 Financial Budget Enforcement: Admission → Reservation → Reconciliation per Attempt This section covers financial budget accounting in microdollars only. TPM rate-limit accounting is described separately in Section 8.3 and operates independently. Blocking exactly at 100% financial spend is impossible before a request completes because output tokens are unknown in advance. The system uses a three-phase approach per provider attempt to prevent budget overruns while supporting concurrent requests safely.

Phase 1 — Pre-request admission: Estimate maximum cost:  $(\text{estimated\_input\_tokens} + \text{max\_output\_tokens}) \times \text{cost per token}$ . If this estimate would exceed remaining budget, reject immediately (HTTP 402).

Phase 2 — Atomic reservation (per attempt): Before each provider call, atomically deduct the estimated maximum cost from the available budget counter in Redis. Each reservation is tied to the `attempt_id`, not just the `request_id`. This prevents two concurrent requests from both seeing sufficient budget and both proceeding.

Phase 3 — Post-attempt financial reconciliation (per attempt): After each provider attempt finishes, reconcile the financial reservation. Note: TPM bucket is NOT touched during this phase — TPM was already charged conservatively at admission.

- Authoritative usage available (provider returned token counts): compute `actual_cost_microdollars = (\text{actual\_input\_tokens} \times \text{cost\_per\_input}) + (\text{actual\_output\_tokens} \times \text{cost\_per\_output})`. Release over- reservation = `estimated\_microdollars - actual\_microdollars` back to the budget counter. If `actual > estimated`, charge the difference.
- Provider failure with clearly zero billable usage (e.g. connection refused before transmission): release the full financial reservation for that attempt.

- Provider failure with unknown billable usage: apply conservative policy — keep full financial reservation (charge estimated cost). Log the policy application. Do NOT assume cost = 0.
- Streaming: accumulate token counts from chunks where the provider includes them. Reconcile when the stream closes or fails. Do NOT buffer the full response for accounting.
- Partial streaming failure: reconcile using partial token count if available; otherwise apply the conservative policy.
- Fallback attempt: receives its own financial reservation. Reconciled independently from the primary attempt.
- Actual cost > estimated cost: charge full actual cost (no cap at estimate).

## 8.5 Request/Attempt Model and Idempotent Accounting

Every gateway request receives a unique request\_id (UUID v4, generated at request ingress). Every provider call receives a unique attempt\_id (UUID v4, generated per call).

request\_id: "req-abc-123" ■■■■ attempt\_id: "att-001" → primary provider, attempt 1 ■■■■ attempt\_id: "att-002" → primary provider, retry ■■■■ attempt\_id: "att-003" → primary provider, retry ■■■■ attempt\_id: "att-004" → fallback provider, attempt 1 Budget reconciliation is idempotent at the attempt level. Reconciliation of a given attempt\_id is tracked by: gateway:reconciled:{request\_id}:{attempt\_id} → TTL = 24 hours If this key is already set, reconciliation for that attempt is a no-op. Reconciling attempt A does not affect attempt B. Each attempt can be reconciled exactly once.

request\_id is propagated to: all log fields, all OTel span attributes, all cost accounting records, and all audit trail entries. attempt\_id is propagated to: all OTel child spans for provider calls, all budget reconciliation records, and structured log entries for each provider call.

## 8.6 Daily and Monthly Budgets

```
gateway:budget:{team_id}:daily:{YYYY-MM-DD} → integer microdollars spent
gateway:budget:{team_id}:monthly:{YYYY-MM} → integer microdollars spent
```

Both limits are independently enforced. The daily key expires at midnight UTC. The monthly key expires at end of month.

## 9. Routing Architecture

Routing policy must NOT be hardwired inside FastAPI endpoint functions. A RoutingEngine/FallbackRouter is an independent, testable component.

### 9.1 Logical Model Tiers

Teams request a quality tier, not a specific model. This allows routing to change without client-side changes.

- fast — lowest-latency, lowest-cost models (e.g. groq/llama-3.3-70b-versatile, openrouter/openrouter-free)
- balanced — mid-range latency/cost (e.g. gemini/gemini-2.5-pro)
- high-quality — highest-capability models (e.g. gemini/gemini-2.5-pro)

### 9.2 RoutingEngine

RoutingEngine.select\_provider(request, team\_config) → ModelConfig considers:

- Requested logical tier
- Team permissions (allowed tiers/providers)
- Provider availability (circuit breaker state from Redis)
- Passive health signals (EWMA latency, error rate)
- Configured fallback chain from routing.yaml

### 9.3 Fallback Chain Configuration

```
config/routing.yaml
```

```
tiers: fast: fallback_chain: - groq/llama-3.3-70b-versatile - gemini/gemini-2.5-flash - openrouter/openrouter-free
balanced: fallback_chain: - groq/llama-3.3-70b-versatile - gemini/gemini-2.5-pro - openrouter/openrouter-free
```



## 10. Retry & Fallback Semantics

### 10.1 Retryable vs Non-Retryable Errors

Retryable Non-Retryable Timeout (ProviderTimeoutError) Authentication failure (401) Connection error Invalid request / bad payload (400) HTTP 429 — honor Retry-After header Unsupported model (400) HTTP 500, 502, 503, 504 Content policy rejection (where provider signals non-retryable)

### 10.2 Exponential Backoff + Jitter

Retry delays use exponential backoff with full jitter:  $\text{delay} = \text{random}(0, \text{base\_delay} \times 2^{\text{attempt}})$ .

Jitter is critical. Without it, all replicas that hit a provider failure simultaneously will retry at the same moment, creating synchronized retry storms that overwhelm a recovering provider. Full jitter spreads retries uniformly over the window, dramatically reducing peak load on recovery.

When a provider returns Retry-After, that value overrides the calculated backoff as a minimum delay. Retry count is configurable per tier (default: 3). Retries are attempted on the same provider before fallback is triggered.

### 10.3 Fallback Behavior and Attempt Accounting

After exhausting retries on the primary provider, the FallbackExecutor tries the next provider in the tier's fallback chain.

Each provider call — whether a retry or a fallback — creates a new ProviderAttempt with its own attempt\_id.

Budget reservation is made per attempt (before the call) and reconciled per attempt (after the call completes or fails). Retries reuse the same provider; fallbacks create a new attempt against a different provider. The attempt\_number increments across retries and fallbacks under the same request\_id.

■ A failed provider attempt does NOT necessarily mean zero billable cost. Apply the conservative policy when authoritative usage is unavailable (Section 8.3).

If no provider in the chain succeeds, a 503 is returned with a clear error body indicating all providers were exhausted.

## 11. Circuit Breaker

### 11.1 States

State Behavior CLOSED Normal operation. Failures counted.

OPEN Provider skipped by RoutingEngine. Requests to this provider immediately route to next in chain. Does NOT return 503 unless no alternative exists.

HALF\_OPEN One probe allowed. Success → CLOSED. Failure → OPEN (cooldown doubles).

### 11.2 Key Design Points

- Circuit state stored in Redis → shared across all gateway replicas.
- OPEN circuit does NOT cause 503 unless no eligible provider remains. The RoutingEngine skips the provider and tries the next in the fallback chain.
- HALF\_OPEN probe is limited to one concurrent probe across all replicas (Redis atomic check-and-set prevents multiple simultaneous hammer-on-recovery).
- Cooldown doubles on each consecutive HALF\_OPEN failure (configurable cap).
- Each probe is a separate ProviderAttempt with its own attempt\_id for tracing and budget purposes.

### 11.3 State Transition Logging

■ Do NOT log giant affected\_teams[] arrays on circuit transitions. The array is unbounded and can flood logs.

Log fields on every circuit breaker state change: provider, model, old\_state, new\_state, reason, timestamp affected\_team\_count (integer — optional, useful for impact assessment)

## 12. Provider Health Monitoring

■ Active probes that make real paid LLM API calls every 30 s are expensive and uninformative. Use PASSIVE signals from real traffic as the primary health signal.

### 12.1 Passive Health Signals

Every gateway request updates provider health state as a side effect:

- Successes: update EWMA latency
- Failures: increment error type counter (429, 5xx, timeout, connection)
- Rolling error rate calculated over the last N requests (configurable window)

### 12.2 EWMA Latency

Exponentially Weighted Moving Average is used instead of simple averages or "P99 of last 5 probes" (which is statistically meaningless with 5 samples).

$\text{ewma\_latency} = \alpha * \text{observed\_latency} + (1 - \alpha) * \text{ewma\_latency}$  Alpha  $\approx 0.1$  gives a smooth signal. Alpha is configurable.

### 12.3 Active Probes

Active probes are used sparingly:

- After provider inactivity (no traffic for configurable period)
- During recovery (HALF\_OPEN state probe in circuit breaker)
- Periodic low-frequency heartbeat (configurable, default every 5 minutes — not 30 s) For MockLLMProvider, active probes are free. For paid providers, active probes use the minimal model and shortest possible prompt.

### 12.4 Health Status Enum

- HEALTHY — error rate  $< 5\%$ , EWMA latency within  $2\times$  baseline
- DEGRADED — error rate  $5\text{--}25\%$  OR EWMA latency  $2\text{--}5\times$  baseline
- DOWN — error rate  $> 25\%$  OR 3 consecutive failures

## 13. Streaming Semantics

### 13.1 Pre-Commitment Failures

Before the first chunk is sent to the client: the gateway may transparently retry or fall back to another provider. The client sees no indication of the failure.

### 13.2 Post-Commitment Failures (Mid-Stream)

■ NEVER silently restart a response from a different provider after streaming has begun. This could produce duplicated, contradictory, or truncated output with no warning to the client.

After the first SSE chunk is committed to the client:

- Terminate the stream cleanly (send a final [DONE] or error chunk per SSE spec).
- Record partial failure in the structured log with `partial_token_count` and failure reason.
- Release/reconcile budget reservation for the current attempt using partial token count if available, else apply the conservative policy.
- Emit metrics and trace the partial failure.
- Do NOT attempt transparent provider fallback.

### 13.3 Streaming and Accounting

The gateway must NOT buffer the entire streaming response merely for accounting purposes. Token counts from streaming chunks (where the provider includes them) should be accumulated incrementally. Final reconciliation for the attempt happens when the stream closes or fails.

## 14. Observability

### 14.1 Structured Logging

Required safe fields in every log line: timestamp, request\_id, attempt\_id, trace\_id, team\_id, logical\_model, provider, served\_model, latency\_ms, retry\_count, fallback\_count, error\_type NEVER log:

- API keys or Authorization headers
- The server-side secret pepper
- Provider credentials
- Full prompt content (default off; can be toggled only for specific debug sessions with explicit configuration)
- Full generated response content (same policy)

### 14.2 Prometheus Metrics & Cardinality

■ team\_id must NOT be placed on high-throughput Prometheus metrics. In production with many tenants,

unbounded cardinality destroys Prometheus performance. Use team\_id only on low-cardinality admin/business metrics.

Operational metrics (bounded labels only): gateway\_requests\_total{provider, model\_tier, status\_class}  
gateway\_request\_duration\_seconds{provider, model\_tier} # Histogram gateway\_tokens\_total{provider, model\_tier, direction} # direction=input|output gateway\_fallback\_total{from\_provider, to\_provider}  
gateway\_circuit\_breaker\_state{provider, model} # Gauge 0=closed,1=half\_open,2=open  
gateway\_rate\_limit\_rejected\_total{limit\_type} # rpm|tpm|budget gateway\_provider\_health{provider, model} # Gauge 0-1  
gateway\_overhead\_ms{provider, model\_tier} # Histogram: measured overhead Per-team business data (stored in Redis/Postgres; NOT Prometheus): Team cost, budget utilisation, and per- team request counts are stored in Redis and Postgres and exposed via the admin API or a low-frequency Prometheus gauge. team\_id is acceptable on a few low-cardinality demo-tier gauges only because the demo has bounded teams.

### 14.3 OpenTelemetry Spans

Root span: llm\_gateway.request (carries request\_id) Child spans in order:

- gateway.auth → key\_valid, team\_id
- gateway.rate\_limit → rpm\_remaining, tpm\_remaining, budget\_remaining\_pct
- gateway.budget\_admission → estimated\_cost\_microdollars, attempt\_id

- gateway.provider\_select → selected\_provider, fallback\_used, circuit\_breaker\_triggered
- gateway.llm\_call → attempt\_id, provider, model, input\_tokens, output\_tokens, latency\_ms, provider\_call\_duration\_ms
- gateway.budget\_reconcile → attempt\_id, actual\_cost\_microdollars, over\_reservation\_released, conservative\_policy\_applied For requests with retries or fallbacks, gateway.llm\_call and gateway.budget\_reconcile appear as sibling child spans, one per attempt\_id. Both request\_id and attempt\_id are propagated as span attributes.

## 14.4 Grafana Dashboards

- Operations: provider health heatmap, error rate by provider, fallback events, circuit breaker state, rate- limit rejections
- Business: per-team daily cost, budget utilisation, top expensive teams, 7-day cost trend (data from admin API)
- Performance: P50/P95/P99 latency by provider, token throughput, gateway overhead (measured)

■ No queue-depth metric appears in any core Grafana dashboard. Queue depth is a stretch-goal concept

only.

## 14.5 Gateway Overhead Measurement

Gateway overhead is defined as an application-level approximation of the time the gateway itself contributes to a request, computed by subtracting measured provider-call durations from total request duration. It is measured using instrumentation, not by subtracting a configured constant.

$\text{gateway\_overhead\_ms} = \text{total\_request\_duration\_ms} - \text{sum}(\text{measured\_provider\_call\_duration\_ms across all attempts})$  Where: measured\_provider\_call\_duration\_ms = wall-clock duration of the actual provider adapter call, timed from just before calling the provider adapter to just after it returns (or raises). Instrumented via OTel span timing on gateway.llm\_call spans.

For retries/fallbacks: sum all attempt durations under the same request\_id.

What this metric captures: gateway routing, auth, rate limiting, budget admission, retry scheduling, budget reconciliation, serialisation, and any instrumentation overhead — all time not spent inside a measured provider adapter call.

Limitations and what this metric does NOT claim to be:

- It is not a measurement of pure gateway CPU cost.

- It may include framework scheduling, event-loop scheduling, and connection-pool handling that occurs outside the measured provider span boundary.
- It may include serialisation/deserialisation that occurs outside the provider timing boundary.
- It includes instrumentation overhead from the timing itself.
- Under high concurrency, event-loop contention may inflate this number beyond steady-state gateway cost.

Note: because the provider network round-trip IS inside `measured_provider_call_duration_ms` (the span wraps the full adapter call), it is subtracted out. This metric does not double-count provider latency. For load tests using `MockLLMProvider`, `provider_call_duration_ms` is the measured duration of the mock adapter call — not the configured `MOCK_LATENCY_MS` constant. This ensures the metric remains valid even when the mock's observed latency diverges from its configured value under load. Load-test reporting uses this formula. The Locust custom reporter reads `measured_provider_duration_ms` from the `X-Provider-Duration-MS` response header (set by gateway instrumentation per request) and computes:  $\text{overhead\_ms} = \text{total\_latency\_ms} - \text{measured\_provider\_duration\_ms}$ . The benchmark report must document the timing boundary clearly.

## 15. API Key Security

### 15.1 Key Design — HMAC-SHA-256 with Server-Side Secret Pepper

API keys are machine-generated high-entropy credentials. Unlike low-entropy human passwords (which require deliberately expensive password hashing such as bcrypt or Argon2 to slow brute-force), API keys with  $\geq 128$  bits of cryptographically secure random entropy are not practically brute-forceable. HMAC-SHA-256 with a server-side secret pepper is therefore appropriate: it is fast for legitimate verification and provides keyed integrity that prevents offline dictionary attacks even if the database is compromised, because the pepper is not stored in the database.

### 15.2 Key Format

- All gateway keys use the prefix `llmgw_` followed by  $\geq 22$  characters of URL-safe base64 random material (providing  $\geq 128$  bits of entropy).
- Admin keys use the prefix `llmgw_admin_`.
- The full key is displayed ONCE at generation time and never stored in plaintext.

### 15.3 Storage Schema (PostgreSQL `api_keys` table)

```
key_id UUID PRIMARY KEY
team_id UUID NOT NULL REFERENCES teams(team_id)
key_prefix VARCHAR(12) NOT NULL -- first 12 chars of llmgw... for fast lookup
hmac_digest BYTEA NOT NULL -- HMAC-SHA-256(pepper, full_key)
created_at TIMESTAMPTZ NOT NULL
expires_at TIMESTAMPTZ
revoked_at TIMESTAMPTZ
```

■ The server-side secret pepper is sourced from environment/secrets configuration and is NEVER stored in the database, source code, or logs.

### 15.4 Verification Algorithm

1. **Validate key format: must start with `llmgw_` (or `llmgw_admin_`).**
2. **Extract lookup prefix (first 12 chars) to find candidate key row(s).**
3. **Load pepper from environment:**

```
pepper = os.environ["API_KEY_PEPPER"] # must be set; fatal if absent
```

#### 4. **Compute:**

```
digest = HMAC-SHA-256(key=pepper, msg=presented_full_api_key)
```

#### 5. **Compare using constant-time comparison:**

```
hmac.compare_digest(digest, stored_hmac_digest)
```

6. Reject if key is expired (`expires_at < now`) or revoked (`revoked_at IS NOT NULL`).



**7. On success, load TeamConfig associated with team\_id.**

### **15.5 Key Lifecycle**

- Keys support revocation (immediate) and rotation (generate new, revoke old).
- No credentials of any kind are hardcoded in source code.
- .env.example contains placeholder variable names only, never real values.

- Admin credentials are separate from team keys and require a separate secret store / env var.

## 16. Health & Readiness Endpoints

Endpoint Behavior GET /health Returns 200 if the process is alive. Used by Docker health check. Fast response; no dependency checks.

GET /ready Returns 200 only if all required dependencies are available (Redis connected, DB connected, config loaded). Returns 503 with reason if not ready.

Both endpoints are used in docker-compose health checks. /ready is used by orchestrators to determine when a new replica may receive traffic.

## 17. Graceful Startup & Shutdown

### 17.1 Startup (FastAPI lifespan)

- Run PostgreSQL migrations; verify schema version (teams and api\_keys tables must exist)
- Initialize HTTP connection pools (httpx async clients per provider)
- Connect to Redis; verify with PING
- Connect to PostgreSQL; verify schema version
- Load and validate configuration (pricing.yaml, routing.yaml, teams)
- Start passive health monitoring background tasks
- Initialize OpenTelemetry tracer and Prometheus metrics
- Log startup complete with version, config hash, and environment

### 17.2 Shutdown

- Stop accepting new requests where practical (graceful drain)
- Allow active streaming responses a bounded graceful-shutdown window (configurable, default 30 s)
- Cancel health monitoring tasks cleanly
- Close all httpx clients
- Close Redis connection pool
- Close PostgreSQL connection pool
- Flush OpenTelemetry span exporters
- Log shutdown complete

## 18. Testing Strategy

### 18.1 Unit Tests

All unit tests use mocks/fakes. No Redis, no database, no network. Coverage targets:  $\geq 80\%$ .

- Provider adapters: normalized response shapes, exception translation
- RoutingEngine: tier resolution, permission checking, health-based selection
- RetryPolicy: retryable classification, backoff calculation, jitter bounds
- Circuit breaker state machine: CLOSED  $\rightarrow$  OPEN  $\rightarrow$  HALF\_OPEN transitions
- Cost calculation: microdollar arithmetic, estimation logic
- Budget admission: reservation, reconciliation, attempt-level idempotency
- HMAC-SHA-256 key verification: valid key, expired key, revoked key, wrong pepper

### 18.2 Redis Integration Tests

■ Use a REAL Redis container for all Lua/concurrency tests. fakeredis does not execute real Lua scripts and cannot prove distributed concurrency correctness.

- Atomic dual-bucket admission: TPM rejection does NOT consume RPM capacity; RPM rejection does NOT consume TPM capacity; both pass  $\rightarrow$  both consumed.
- Conservative TPM test: admit a request with estimated\_tpm\_tokens = 5000 (input=1000, max\_output=4000); after provider returns actual\_output=500; verify TPM bucket still shows 5000 tokens consumed, NOT 1500. No refund operation is performed on the TPM bucket.
- Token bucket Lua script: concurrent consume, refill, exact capacity enforcement
- RPM + TPM dual enforcement under concurrent load (200 coroutines)
- Budget reservation: concurrent requests, no double-spend
- Attempt-level idempotent reconciliation: duplicate (request\_id, attempt\_id)  $\rightarrow$  no double charge
- Multi-attempt reconciliation: attempt A and attempt B under same request\_id reconcile independently
- Reconciliation of attempt A does not prevent reconciliation of attempt B
- Circuit breaker distributed state: HALF\_OPEN probe limit across replicas

### 18.3 Full Integration Tests

FastAPI + real Redis + test database + MockLLMProvider. No real provider API calls.

Test Case What It Verifies MockLLMProvider Mode test\_auth\_valid\_key 200 + correct response shape normal  
test\_auth\_invalid\_key 401 normal test\_model\_tier\_not\_allowed 403 normal

test\_rpm\_rate\_limit Exactly 0 over-limit requests slip normal + latency through test\_tpm\_rate\_limit TPM bucket enforced normal independently test\_tpm\_conservative\_no\_refund After completion with normal actual\_output < max\_output\_tokens, TPM bucket charged estimated amount; no refund test\_rpm\_tpm\_atomic\_tpm\_fail TPM rejection → RPM capacity normal unchanged test\_rpm\_tpm\_atomic\_rpm\_fail RPM rejection → TPM capacity normal unchanged test\_budget\_block 402 when budget exhausted normal test\_budget\_idempotency\_per\_attempt Same (request\_id, attempt\_id) normal reconciled twice → no double charge test\_budget\_multi\_attempt Attempt A and B reconcile normal independently test\_fallback\_on\_500 Response from fallback provider HTTP 500 → normal test\_fallback\_attempt\_ids Primary and fallback attempts HTTP 500 → normal have distinct attempt\_ids test\_circuit\_breaker\_opens OPEN state after N failures; no HTTP 500 × N direct provider calls test\_circuit\_breaker\_skips\_to\_fallback 200 from fallback while primary OUTAGE + normal OPEN test\_circuit\_breaker\_recovers CLOSED after successful RECOVERY HALF\_OPEN probe test\_all\_providers\_down 503 with clear error OUTAGE all test\_streaming\_passthrough SSE chunks arrive before stream streaming closes test\_mid\_stream\_failure Stream terminates; no silent streaming → 500 mid- restart stream test\_conservative\_policy\_applied Failed attempt with unknown CONNECTION\_FAIL usage charges reservation test\_admin\_limit\_update New RPM reflected within 500 ms normal test\_audit\_log\_entry Admin action creates Postgres normal audit row

## 18.4 Load Tests

Load tests use Locust with MockLLMProvider (configurable simulated latency). No paid API calls.

#### Benchmark conditions:

- Total requests:  $\geq 5,000$
- Concurrent users: 200 (ramp 10  $\rightarrow$  200 over 60 s, hold 120 s)
- Duration: 3 minutes Traffic patterns (not priorities):
- InteractiveUser: short prompts (50–100 tokens), 150 users — represents interactive chat workload
- LargeContextUser: long prompts (1,000–4,000 tokens), 50 users — represents document/context-heavy workload These represent different traffic patterns with different TPM characteristics. They are NOT different queue priorities (priority queues are a stretch goal only). Measurements:
- Throughput (RPS)
- P50 gateway overhead (measured\_provider\_call\_duration\_ms subtracted per request)
- P95 gateway overhead
- P99 gateway overhead
- Error rate
- Rate-limit correctness (zero slip-through)
- Fallback success rate
- Budget violations

■ README and CV must contain ACTUAL measured numbers from your own load test run. Never invent or massage results to satisfy the PRD targets.

## 19. CI Pipeline (GitHub Actions)

- Dependency installation (pip with pyproject.toml)
- Ruff linting
- Ruff format check
- mypy or pyright type checking
- Unit tests (pytest, no external services)
- Redis integration tests (real Redis via docker service)
- Coverage report ( $\geq 80\%$  target)
- Docker image build
- Optional: lightweight dependency/security scan (pip-audit or safety) No provider API credentials are required in CI. All tests use MockLLMProvider.



## 20. Repository Structure

```
llm-gateway/
■
■■■ src/
■ ■■■ gateway/
■ ■■■ api/ # FastAPI routers and endpoint definitions
■ ■■■ domain/ # ChatMessage, ProviderAttempt, models, exceptions
■ ■■■ auth/ # API key validation (HMAC-SHA-256), team lookup
■ ■■■ providers/
■ ■ ■■■ base.py # LLMPProvider ABC
■ ■ ■■■ mock.py # MockLLMProvider
■ ■ ■■■ groq.py
■ ■ ■■■ gemini.py
■ ■ ■■■ openrouter.py
■ ■■■ routing/ # RoutingEngine, FallbackExecutor
■ ■■■ rate_limiting/ # Atomic dual-bucket Lua, RateLimiter
■ ■■■ budget/ # BudgetManager, admission, per-attempt reconciliation
■ ■■■ resilience/
■ ■ ■■■ retry.py
■ ■ ■■■ circuit_breaker.py
■ ■ ■■■ health.py
■ ■■■ observability/ # OTel, Prometheus metrics, structlog config
■ ■■■ admin/ # Admin API, audit log
■ ■■■ config/ # YAML loaders, validated config models
■ ■■■ main.py # FastAPI app factory + lifespan
■
■■■ database/
■ ■■■ migrations/
■ ■■■ 001_initial.sql # teams, api_keys (required before Phase 3)
■
■■■ tests/
■ ■■■ unit/
■ ■■■ integration/
■ ■■■ load/ # Locust locustfile.py
■
■■■ config/
■ ■■■ pricing.yaml # Provider pricing (Decimal strings)
■ ■■■ routing.yaml # Tier fallback chains
■ ■■■ teams/ # Per-team config YAMLs (dev/test only)
■
■■■ infra/
■ ■■■ docker-compose.yml
■ ■■■ prometheus.yml
■ ■■■ grafana/ # Dashboard + datasource provisioning
■
■■■ docs/
■■■ scripts/
■■■ .github/workflows/
■■■ Dockerfile
■■■ docker-compose.yml
■■■ pyproject.toml
■■■ .env.example
■■■ README.md
```

## 21. Development Phases

Phase Name Key Deliverable Phase 0 Architecture + Repo structure, pyproject.toml, docker-compose skeleton, CI skeleton stub, .env.example Phase 1 Domain models + All domain models (incl. ProviderAttempt), LLMProvider ABC, interface + Mock MockLLMProvider with all simulation modes, full unit tests Phase 2 PostgreSQL database/migrations/001\_initial.sql (teams, api\_keys), DB foundation connection infrastructure, migration execution on startup Phase 3 Real providers + Groq/Gemini/OpenRouter adapters, POST unified API + /v1/chat/completions, streaming passthrough streaming Phase 4 Auth + Redis rate HMAC-SHA-256 API key auth (requires Phase 2 DB), atomic dual- limiting bucket Lua, RPM + TPM enforcement, /health + /ready Phase 5 Cost + budget pricing.yaml, microdollar accounting, per-attempt admission + enforcement reservation + reconciliation, attempt-level idempotency Phase 6 Retry + routing + RoutingEngine, RetryPolicy, FallbackExecutor (with attempt\_id fallback + circuit tracking), CircuitBreaker, passive health signals breaker + health Phase 7 Observability Structured JSON logging (with attempt\_id), OTel + Jaeger (attempt spans), Prometheus metrics (measured overhead), Grafana dashboards Phase 8 Admin + security Admin API, audit log, key lifecycle, Postgres audit trail (via later hardening migration if not in 001) Phase 9 Integration + load Full integration test suite (incl. atomic admission tests), Locust tests + Docker + CI load test (measured overhead), docker-compose stack, GitHub Actions CI Phase 10 Docs + portfolio README, architecture diagram (Mermaid), benchmark report with actual numbers, CV bullets, interview prep

## 22. Stretch Goals (Post-Core)

The following features should NOT block completion of the core project. Implement after Phase 10 is complete.

- Priority queues (interactive vs batch) — consumers may request elevated scheduling; requires a queue depth metric and associated Grafana panel
- Request enrichment (system-prompt prefix, compliance suffix injection)
- Regex content filtering (return 400 on pattern match)
- Slack budget warning webhooks (80% threshold notification)
- Configuration hot reload without restart
- Admin dashboard UI
- Prompt caching for repeated identical requests

■ Priority queues are a STRETCH GOAL ONLY. No priority semantics, queue depth metrics, or queue-related

Grafana panels exist in the core implementation.

## 23. GitHub Copilot Implementation Prompts

All prompts below are designed for GitHub Copilot Pro Agent/Ask mode. Each prompt implements ONE concept.

Execute them sequentially. Do not combine prompts. Each prompt should be completable in one Copilot session and its output should be testable before the next prompt begins.

■ These prompts are sized for Copilot Pro context limits. Larger prompts risk incomplete or inconsistent

output. Resist the urge to combine steps.

■ **DEPENDENCY RULE:** Every prompt depends only on files, schemas, classes, and infrastructure created by

earlier prompts. The database migration (Prompt 2.1) must exist before API-key auth (Prompt 4.1).

Phase 0 — Architecture & Skeleton

### ***Prompt 0.1: Repository skeleton***

Create the project scaffold for an LLM gateway (Python 3.11+, FastAPI, src layout). Create:

- pyproject.toml with [project] metadata, dependencies (fastapi, uvicorn, pydantic, httpx, redis, asyncpg, structlog, opentelemetry-sdk), and dev-dependencies (pytest, pytest-asyncio, ruff, mypy, locust).
- Directory structure:  
src/gateway/{api, domain, auth, providers, routing, rate\_limiting, budget, resilience, observability, admin, config}/\_\_init\_\_.py
- database/migrations/ directory
- tests/{unit, integration, load}/\_\_init\_\_.py
- config/pricing.yaml and config/routing.yaml as empty stubs with a comment explaining their purpose.
- .env.example with placeholder keys: REDIS\_URL, DATABASE\_URL, ADMIN\_SECRET\_KEY, API\_KEY\_PEPPER, OTEL\_EXPORTER\_OTLP\_ENDPOINT.
- .github/workflows/ci.yml stub that runs pytest. Do not write any implementation code yet. Just the scaffold.

### ***Prompt 0.2: Docker Compose skeleton***

Create an infra/docker-compose.yml that defines the following services: gateway (placeholder build), redis:7- alpine, postgres:15-alpine, prometheus (prom/prometheus), grafana (grafana/grafana), jaeger (jaegertracing/all-in-one).

Add health checks for redis and postgres. Add named volumes for postgres-data and grafana-data.

Add environment variable placeholders using .env for the gateway service. Include API\_KEY\_PEPPER as a required env var (placeholder value in .env.example; fatal if absent at startup).

Do not configure Prometheus scrape targets or Grafana provisioning yet — add TODO comments for those.

Phase 1 — Domain Models, Interface & MockLLMProvider

### **Prompt 1.1: Domain models**

In `src/gateway/domain/models.py`, define these Pydantic v2 dataclasses with full type hints and docstrings:

- `ChatMessage`(role: Literal["system", "user", "assistant"], content: str)
- `ChatCompletionRequest`(messages: list[ChatMessage], model\_tier: str, max\_tokens: int, temperature: float, stream: bool, team\_id: str, request\_id: str)
- `ChatCompletionResponse`(text: str, input\_tokens: int, output\_tokens: int, latency\_ms: float, model\_id: str, provider: str, finish\_reason: str, request\_id: str)
- `Usage`(input\_tokens: int, output\_tokens: int, cost\_microdollars: int)
- `StreamingChunk`(delta: str, finish\_reason: str | None, index: int)
- `ModelConfig`(provider: str, model\_id: str, quality\_tier: str, max\_tokens: int, cost\_per\_input\_token\_microdollars: int, cost\_per\_output\_token\_microdollars: int)
- `AttemptOutcome`(Enum): SUCCESS | RETRYABLE\_FAILURE | NON\_RETRYABLE\_FAILURE | STREAMING\_PARTIAL
- `ProviderAttempt`(attempt\_id: str, request\_id: str, attempt\_number: int, provider: str, model: str, estimated\_cost\_microdollars: int, actual\_cost\_microdollars: int | None, outcome: AttemptOutcome | None) Write unit tests in `tests/unit/test_domain_models.py` verifying construction and field validation.

### **Prompt 1.2: Normalized provider exceptions**

In `src/gateway/domain/exceptions.py`, define these exception classes with docstrings:

- `ProviderError`(message, provider, model, status\_code) — base
- `ProviderTimeoutError` — retryable
- `ProviderRateLimitError`(retry\_after\_seconds: float | None) — retryable
- `ProviderUnavailableError`(status\_code: int) — retryable
- `ProviderAuthenticationError` — non-retryable
- `InvalidRequestError` — non-retryable
- `InvalidProviderResponseError` — non-retryable Add a `is_retryable(exc: ProviderError) -> bool` helper function. Write unit tests in `tests/unit/test_exceptions.py`.

### **Prompt 1.3: LLMProvider interface**

In `src/gateway/providers/base.py`, define:

- `LLMProvider`(ABC) with abstract async methods: `complete(request) -> ChatCompletionResponse`; `stream(request) -> AsyncIterator[StreamingChunk]`; `health_check() -> ProviderHealth`
- `ProviderHealth`(status: HealthStatus, latency\_ms: float | None, error: str | None)
- `HealthStatus`(Enum): HEALTHY, DEGRADED, DOWN In `src/gateway/providers/registry.py`, define `ProviderRegistry` that maps (provider, model\_id) -> `LLMProvider`. Write unit tests confirming the registry raises on unknown providers.

### **Prompt 1.4: MockLLMProvider — normal completion + streaming**

In `src/gateway/providers/mock.py`, implement `MockLLMProvider(LLMProvider)` with:

- Constructor args: `mode` (default `NORMAL`), `latency_ms` (default 50), `response_text` (default "Mock response.")
- `complete()`: sleeps for `latency_ms` then returns `ChatCompletionResponse` with realistic token counts.
- `stream()`: yields `StreamingChunk` tokens word-by-word with configurable latency per chunk.
- `health_check()`: returns `HEALTHY`. Write unit tests in `tests/unit/test_mock_provider.py` for both `complete()` and `stream()`.

### **Prompt 1.5: MockLLMProvider — failure modes**

Extend `MockLLMProvider` to support these additional modes (pass as `mode=` enum argument):

- `RATE_LIMITED`: raises `ProviderRateLimitError(retry_after_seconds=2.0)`
- `SERVER_ERROR(status_code)`: raises `ProviderUnavailableError(status_code)`
- `TIMEOUT`: raises `ProviderTimeoutError` after simulated delay
- `CONNECTION_FAIL`: raises `ProviderError` immediately (simulates zero billable usage)
- `MALFORMED_RESPONSE`: raises `InvalidProviderResponseError`
- `OUTAGE`: always raises `ProviderUnavailableError(503)`
- `RECOVERY(recover_after_calls)`: raises `ProviderUnavailableError` for first N calls, then returns normally Write unit tests for every failure mode.

■ This phase MUST be completed before Phase 4 (Auth). The `api_keys` table must exist before API-key

authentication is implemented.

### **Prompt 2.1: PostgreSQL initial migration**

Create `database/migrations/001_initial.sql` with tables: -- teams `CREATE TABLE teams ( team_id UUID PRIMARY KEY DEFAULT gen_random_uuid(), name TEXT NOT NULL, created_at TIMESTAMPTZ NOT NULL DEFAULT now() );` -- api\_keys `CREATE TABLE api_keys ( key_id UUID PRIMARY KEY DEFAULT gen_random_uuid(), team_id UUID NOT NULL REFERENCES teams(team_id), key_prefix VARCHAR(12) NOT NULL, hmac_digest BYTEA NOT NULL, created_at TIMESTAMPTZ NOT NULL DEFAULT now(), expires_at TIMESTAMPTZ, revoked_at TIMESTAMPTZ );` `CREATE INDEX idx_api_keys_prefix ON api_keys(key_prefix);`

In `src/gateway/config/database.py`, implement `DatabaseConfig` that:

- Reads `DATABASE_URL` from environment.
- On startup: connects via `asyncpg`, applies any unapplied migrations from `database/migrations/` in order.
- Exposes `get_connection()` for use by auth and admin modules. Write a test that applies the migration to a test Postgres instance and verifies the schema (teams and api\_keys tables exist with correct columns). Phase 3 — Real Providers + Unified API + Streaming

### ***Prompt 3.1: Pricing config loader***

In `src/gateway/config/pricing.py`:

- Define a `PricingConfig` Pydantic model that reads `config/pricing.yaml`.
- Pricing values must be loaded as Python Decimal (from `decimal import Decimal`), not float.
- Provide a function `get_model_config(provider, model_id) -> ModelConfig` that converts Decimal pricing to integer microdollars (multiply by 1,000,000, round to int).
- Add a `cost_estimate_microdollars(model_config, input_tokens, max_output_tokens) -> int` helper. This computes:  $(input\_tokens \times cost\_per\_input) + (max\_output\_tokens \times cost\_per\_output)$  as the conservative pre-request estimate. Write unit tests for the loader and cost estimation with known values from the example `pricing.yaml`.

### ***Prompt 3.2: Groq provider adapter***

In `src/gateway/providers/groq.py`, implement `GroqProvider(LLMProvider)`:

- `complete()`: calls the Groq async client, maps response to `ChatCompletionResponse`.
- `stream()`: calls Groq SDK streaming, yields `StreamingChunk` for each delta.
- `health_check()`: a lightweight call that does NOT make a paid completion request (use `models.list` or a known fast endpoint).
- Translate all Groq exceptions to the normalized exceptions from `domain/exceptions.py`. Write unit tests that mock the Groq client and verify exception translation for: `AuthenticationError`, `RateLimitError`, `APIConnectionError`, `APIStatusError(500)`.

### ***Prompt 3.3: Gemini provider adapter***

In `src/gateway/providers/gemini.py`, implement `GeminiProvider(LLMProvider)`:

- `complete()`: calls the Google Gemini async client, maps response to `ChatCompletionResponse`.
- `stream()`: handles Gemini streaming, yields `StreamingChunk`.
- `health_check()`: lightweight check (do not make a paid completion).
- Translate Gemini-specific exceptions to normalized exceptions. Write unit tests mocking the Gemini client with same failure coverage as 3.2.

### **Prompt 3.4: OpenRouter provider adapter**

In `src/gateway/providers/openrouter.py`, implement `OpenRouterProvider(LLMProvider)`:

- Uses `httpx.AsyncClient` to call the OpenRouter REST API.
- `complete()`: POST `/api/chat`, map response.
- `stream()`: POST `/api/chat` with `stream=true`, yield `StreamingChunk` from NDJSON lines.
- `health_check()`: GET `/api/tags` to verify service is running.
- Translate `httpx/HTTP` errors to normalized exceptions. Write unit tests using `respx` to mock the HTTP calls.

### **Prompt 3.5: FastAPI app factory + POST /v1/chat/completions (non-streaming)**

In `src/gateway/main.py`, create a FastAPI app with lifespan. In `src/gateway/api/completions.py`:

- Implement POST `/v1/chat/completions` that accepts OpenAI-compatible request body (messages, model, max\_tokens, temperature, stream).
- For now: stub auth (always allowed), use `MockLLMProvider` in NORMAL mode, return `ChatCompletionResponse` as JSON.
- Add `X-Provider-Used`, `X-Request-ID`, and `X-Attempt-ID` response headers. Write integration tests using `httpx.AsyncClient` verifying: 200 response shape, correct headers.

### **Prompt 3.6: Streaming endpoint**

Extend POST `/v1/chat/completions` to handle `stream=true`:

- Use FastAPI `StreamingResponse` with `media_type="text/event-stream"`.
- Call `provider.stream()` and yield each chunk as an SSE data: line.
- After the stream closes, collect final token counts for logging (do NOT buffer the full text).
- Mid-stream failure: catch `ProviderError` after first chunk sent, emit a final SSE error event, close stream cleanly. Write integration tests: verify SSE chunks arrive incrementally; verify mid-stream failure terminates cleanly without 500. Phase 4 — Auth + Redis Rate Limiting

■ Depends on Phase 2 (database/migrations/001\_initial.sql must exist and the `teams/api_keys` tables must

be present).

### **Prompt 4.1: API key auth + team config (HMAC-SHA-256)**

In `src/gateway/auth/`:

- Define `TeamConfig(Pydantic)`: `team_id`, `allowed_tiers`, `rate_limit_rpm`, `rate_limit_tpm`, `daily_budget_microdollars`, `monthly_budget_microdollars`.
- Implement API key verification using HMAC-SHA-256 with a server-side secret pepper (`API_KEY_PEPPER` env var).



- Key format: lmgw\_ prefix +  $\geq 22$  URL-safe base64 chars ( $\geq 128$  bits entropy).
- Verification: validate format  $\rightarrow$  extract key\_prefix (first 12 chars)  $\rightarrow$  query api\_keys table  $\rightarrow$  compute HMAC-SHA256(pepper, presented\_key)  $\rightarrow$  constant-time compare with stored hmac\_digest  $\rightarrow$  check expires\_at and revoked\_at.
- FastAPI dependency get\_team\_config(x\_api\_key: str = Header(...))  $\rightarrow$  TeamConfig: returns 401 on invalid key.
- NEVER log the API key, the pepper, or the computed digest. Write unit tests for key verification logic. Write integration tests for 401 on missing/invalid key. Ensure the test DB has the api\_keys table from migration 001.

#### **Prompt 4.2: Health and readiness endpoints**

In src/gateway/api/health.py, add:

- GET /health: returns 200 {status: "ok"} immediately. No dependency checks.
  - GET /ready: checks Redis PING and DB connection. Returns 200 {status: "ready"} or 503 {status: "not\_ready", reason: "..."}.
- Add both to docker-compose healthcheck configurations. Write unit tests for both endpoints using mocked Redis and DB.

#### **Prompt 4.3: Atomic dual-bucket token bucket Lua script**

In src/gateway/rate\_limiting/lua\_scripts.py, write a Redis Lua script that implements atomic dual-bucket admission (see Section 7.4):

- Single Lua script receives two bucket keys (RPM and TPM) and all parameters.
- Refills both buckets atomically.
- Checks both: if either lacks capacity, consumes NOTHING from either and returns allowed=0 with retry\_after\_ms.
- If both have capacity: consumes 1 RPM token and estimated\_tpm\_tokens TPM tokens; persists both bucket states atomically; returns allowed=1 with remaining\_rpm and remaining\_tpm.
- IMPORTANT: The Lua script has no refund/return path. TPM tokens consumed at admission are permanently consumed for rate-limiting purposes. There is no post-execution write-back to the TPM bucket. See Section 8.3 for the design rationale. Also implement a single-bucket version of the script for isolated unit testing/reuse (single\_bucket\_lua\_script). The dual-bucket script is what the RateLimiter uses for actual admission. Write tests using a REAL Redis container. Required tests:
- test\_tpm\_reject\_does\_not\_consume\_rpm: fill RPM fully, exhaust TPM  $\rightarrow$  admission fails  $\rightarrow$  RPM unchanged.
- test\_rpm\_reject\_does\_not\_consume\_tpm: exhaust RPM, fill TPM fully  $\rightarrow$  admission fails  $\rightarrow$  TPM unchanged.
- test\_both\_pass\_both\_consumed: both buckets have capacity  $\rightarrow$  both consumed by correct amounts.
- test\_concurrent\_200\_coroutines: 200 concurrent admissions at  $2\times$  RPM cap  $\rightarrow$  exactly 0 slip through.
- test\_conservative\_tpm\_no\_refund: admit a request with estimated\_tpm\_tokens=5000 (input=1000, max\_output=4000). After the request completes and MockLLMProvider returns actual\_output=500,

read the TPM bucket state directly from Redis. Assert the bucket shows 5000 tokens consumed, NOT 1500. No second Lua operation is called on the TPM bucket after provider execution.

#### ***Prompt 4.4: RateLimiter class***

In `src/gateway/rate_limiting/limiter.py`, implement `RateLimiter`:

- Constructor: `redis_client`, `team_id`, `limit_rpm`, `limit_tpm`.
- `async check_and_consume(estimated_tpm_tokens: int) -> RateLimitResult(allowed, remaining_rpm, remaining_tpm, retry_after_ms)`.
- Calls the dual-bucket Lua script from Prompt 4.3. `estimated_tpm_tokens = estimated_input_tokens + max_output_tokens` (computed by caller before this call).
- If rejected: returns `allowed=False` with `retry_after_ms`. Write Redis integration tests verifying both RPM and TPM rejection paths leave the other bucket unchanged.

#### ***Prompt 4.5: Rate limit FastAPI integration***

Wire `RateLimiter` into the completions endpoint as a FastAPI dependency:

- After auth, before routing: call `RateLimiter.check_and_consume(estimated_input_tokens + request.max_tokens)`.
- On rejection: return HTTP 429 with `Retry-After` header and JSON body `{error: "rate_limited", retry_after_ms: N}`. Write integration tests: valid key under limit → 200; same key over RPM limit → 429 with `Retry-After`; same key with TPM exhausted → 429 with `Retry-After`. Phase 5 — Cost Calculation + Budget Enforcement

#### ***Prompt 5.1: Budget manager: per-attempt reservation***

In `src/gateway/budget/manager.py`, implement `BudgetManager`:

- `async reserve(team_id, request_id, attempt_id, estimated_microdollars) -> ReservationResult(allowed, reservation_id)`.
- Uses Redis atomic Lua to: check daily + monthly budget remaining, deduct `estimated_microdollars` from both, store reservation record keyed by `(request_id, attempt_id)`.
- Redis keys: `gateway:budget:{team_id}:daily:{YYYY-MM-DD}` and `gateway:budget:{team_id}:monthly:{YYYY-MM}`.
- Returns `allowed=False` (HTTP 402) if budget insufficient. Write Redis integration tests for: normal reservation, budget exhaustion, concurrent reservations → no double-spend.

#### ***Prompt 5.2: Budget manager: per-attempt reconciliation + idempotency***

Extend `BudgetManager`:

- `async reconcile(team_id, request_id, attempt_id, actual_microdollars: int | None, conservative: bool = False) -> None`.
- Idempotency key: `gateway:reconciled:{request_id}:{attempt_id}` with `TTL=86400`. If already set, return immediately.
- If `actual_microdollars` is provided (authoritative usage): `release over-reservation = estimated - actual`. If `actual > estimated`, charge difference.
- If `actual_microdollars` is `None` and `conservative=True`: keep full reservation (charge estimated). Log that conservative policy was applied.
- If `actual_microdollars` is `None` and `conservative=False`: release full reservation (clearly zero usage, e.g. connection refused before transmission). Write Redis integration tests for:
  - `test_reconcile_attempt_idempotent`: same (`request_id`, `attempt_id`) reconciled twice → second call is no-op.
  - `test_reconcile_two_attempts_independent`: `attempt_id_1` and `attempt_id_2` under same `request_id` reconcile independently.
  - `test_reconcile_a_does_not_block_b`: reconciling attempt A does not prevent reconciling attempt B.
  - `test_conservative_policy`: `None actual + conservative=True` → full reservation retained.

### ***Prompt 5.3: Wire budget into completion endpoint***

In the completions endpoint, after rate limiting:

- Generate `attempt_id = uuid4()` for each provider call.
- Call `BudgetManager.reserve(team_id, request_id, attempt_id, estimated_cost)` before the provider call. On failure → 402.
- After provider response (or error): call `BudgetManager.reconcile()` in a finally block so it always runs.
- Pass both `request_id` and `attempt_id` through the entire flow.
- On `CONNECTION_FAIL`: reconcile with `actual=None`, `conservative=False` (clearly zero usage).
- On provider failure with unknown usage: reconcile with `actual=None`, `conservative=True`. Write integration tests: budget block at 100% → 402; reconciliation called even on provider failure; conservative policy applied when usage unknown. Phase 6 — Retry + Routing + Fallback + Circuit Breaker + Health

### ***Prompt 6.1: RetryPolicy***

In `src/gateway/resilience/retry.py`, implement `RetryPolicy`:

- Constructor: `max_retries` (default 3), `base_delay_s` (default 0.5).
- `async execute(fn: Callable, ...) -> T`: calls `fn`, on `ProviderError` checks `is_retryable()`. If retryable and retries remaining: sleep with exponential backoff + full jitter. Honor `retry_after_seconds` from `ProviderRateLimitError`.
- On non-retryable error or retries exhausted: re-raise.

- Each retry creates a new `ProviderAttempt` (`attempt_id`, `request_id`, `attempt_number`). The caller is responsible for generating `attempt_id` per call. Write unit tests: retryable error → retries; non-retryable → immediate raise; Retry-After honored; jitter within expected bounds.

### ***Prompt 6.2: CircuitBreaker state machine (in-memory)***

In `src/gateway/resilience/circuit_breaker.py`, implement `CircuitBreaker(provider, model)`:

- States: `CLOSED`, `OPEN`, `HALF_OPEN` (enum).
- In-memory state machine only (no Redis yet).
- `CLOSED`: count failures; after `failure_threshold` in `window_seconds` → `OPEN`.
- `OPEN`: raise immediately without calling provider.
- `HALF_OPEN`: allow one probe call; success → `CLOSED`; failure → `OPEN` (cooldown doubles). Write unit tests for all `CLOSED→OPEN→HALF_OPEN→CLOSED` and `HALF_OPEN→OPEN` transitions.

### ***Prompt 6.3: CircuitBreaker Redis state***

Extend `CircuitBreaker` to store state in Redis so all gateway replicas share it:

- Redis key: `gateway:cb:{provider}:{model}` → JSON `{state, failure_count, opened_at, cooldown_s}`.
- `HALF_OPEN` probe exclusivity: use Redis `SET NX` with short TTL to allow only ONE probe across all replicas simultaneously.
- Atomic state transitions using Lua or `WATCH/MULTI`. Write Redis integration tests: two concurrent replicas in `HALF_OPEN` → only one probe fires.

### ***Prompt 6.4: RoutingEngine***

In `src/gateway/routing/engine.py`, implement `RoutingEngine`:

- Constructor: `provider_registry`, `circuit_breakers`, `health_store`, `routing_config`.
- `select_provider(request, team_config) -> ModelConfig`: resolves logical tier, checks team permissions, iterates fallback chain, skips providers whose circuit is `OPEN` or health is `DOWN`.
- Returns first available provider. Write unit tests with mocked circuit breakers and health: all healthy → first in chain; first `OPEN` → second; all `OPEN` → raises `NoAvailableProviderError`.

### ***Prompt 6.5: FallbackExecutor with per-attempt tracking***

In `src/gateway/routing/fallback.py`, implement `FallbackExecutor`:

- `async execute(request, team_config) -> ChatCompletionResponse`: uses `RoutingEngine` to select provider, wraps call in `RetryPolicy`.
- Each provider call generates a new `attempt_id`. The `attempt_number` increments across retries and fallbacks.
- Before each provider call: `BudgetManager.reserve(attempt_id=...)`.
- After each provider call (success or failure): `BudgetManager.reconcile(attempt_id=...)` in a finally block.

- On exhausted retries: re-selects next provider from chain (new attempt\_id). Repeats until response or no providers remain. Write integration tests: primary 500 × 3 retries → fallback → success. Assert: primary and fallback have distinct attempt\_ids. Assert: each attempt was independently reserved and reconciled.

### ***Prompt 6.6: Integration: outage and fallback end-to-end***

Wire FallbackExecutor into the completions endpoint. Add a test using MockLLMProvider modes:

- Register two providers: "primary" (OUTAGE mode), "fallback" (NORMAL mode).
- Send request → assert response comes from "fallback" provider.
- Assert circuit breaker for "primary" is OPEN after test.
- Assert 200 returned to client (not 503).
- Assert fallback attempt has a distinct attempt\_id from the failed primary attempt.

### ***Prompt 6.7: Integration: all providers unavailable***

Add integration test:

- Register two providers, both in OUTAGE mode.
- Send request → assert 503 with JSON body explaining all providers exhausted.
- Assert budget reservation was released (or conservative policy applied) for all attempts.

### ***Prompt 6.8: Passive provider health signals***

In src/gateway/resilience/health.py, implement ProviderHealthStore:

- On every provider call result, update in Redis: EWMA latency (alpha=0.1), rolling error counts by type.
- get\_health(provider, model) -> ProviderHealth: compute HealthStatus from error rate + EWMA latency thresholds.
- Active probe: lightweight check used after inactivity or during recovery (configurable frequency, default every 5 min). Write unit tests for EWMA calculation and health status thresholds.

### ***Prompt 6.9: Streaming failure semantics test***

Write integration tests for mid-stream provider failure:

- MockLLMProvider configured to yield 3 chunks then raise ProviderUnavailableError.
- Assert: client receives 3 chunks then a final error SSE event (not a silent restart).
- Assert: no second provider is silently invoked after streaming begins.
- Assert: budget reservation is reconciled for the attempt with partial token count. Phase 7 — Structured Logging + OpenTelemetry + Prometheus + Grafana

### **Prompt 7.1: Structured logging setup**

In `src/gateway/observability/logging.py`, configure structlog:

- JSON output with fields: `timestamp`, `level`, `request_id`, `attempt_id`, `trace_id`, `team_id`, `provider`, `model`, `latency_ms`, `retry_count`, `fallback_count`, `error_type`.
- Add a `RedactingProcessor` that removes any field matching: `api_key`, `authorization`, `credential`, `password`, `prompt`, `completion`, `pepper`.
- FastAPI middleware that adds `request_id` and `trace_id` to structlog context at request start; `attempt_id` is added at the point of each provider call. Write unit tests for the redacting processor with various field names including "pepper".

### **Prompt 7.2: OpenTelemetry instrumentation**

In `src/gateway/observability/tracing.py`:

- Initialize OTel tracer "llm-gateway", exporting to OTLP endpoint from env.
- Create root span `llm_gateway.request` (carries `request_id`) with child spans: `gateway.auth`, `gateway.rate_limit`, `gateway.budget_admission`, `gateway.provider_select`.
- For each provider attempt, create a child span `gateway.llm_call` with attributes: `attempt_id`, `provider`, `model`, `input_tokens`, `output_tokens`, `latency_ms`, `provider_call_duration_ms` (measured duration of the adapter call).
- Create a sibling child span `gateway.budget_reconcile` with attributes: `attempt_id`, `actual_cost_microdollars`, `over_reservation_released`, `conservative_policy_applied`.
- Propagate `trace_id` into structlog fields.
- On error: record exception, set span status `ERROR`. Write a test using in-memory `SpanExporter` asserting all spans exist and required attributes are present, including `attempt_id` on provider-level spans.

### **Prompt 7.3: Prometheus metrics**

In `src/gateway/observability/metrics.py`, define (using `prometheus_client`): `gateway_requests_total` `Counter{provider, model_tier, status_class}` `gateway_request_duration_seconds` `Histogram{provider, model_tier}` `gateway_tokens_total` `Counter{provider, model_tier, direction}` `gateway_fallback_total` `Counter{from_provider, to_provider}` `gateway_circuit_breaker_state` `Gauge{provider, model}` `gateway_rate_limit_rejected_total` `Counter{limit_type} # rpm|tpm|budget` `gateway_provider_health` `Gauge{provider, model}` `gateway_overhead_ms` `Histogram{provider, model_tier}` # computed as: `total_request_duration - sum(provider_call_durations)` Wire into completions endpoint middleware. Add `/metrics` endpoint for Prometheus scraping.

Write a test asserting metrics increment correctly, and that `gateway_overhead_ms` is computed from measured durations not from configured mock latency.

### **Prompt 7.4: Grafana provisioning**

Create `infra/grafana/provisioning/`:

- `datasources/prometheus.yml`: auto-configure Prometheus datasource.

- `datasources/jaeger.yml`: auto-configure Jaeger datasource.
- `dashboards/ops.json`: Operations dashboard (provider health, error rate, fallback events, circuit breaker, rate-limit rejections).
- `dashboards/performance.json`: Performance dashboard (P50/P95/P99 latency by provider, token throughput, gateway overhead using `gateway_overhead_ms`).
- `dashboards/business.json`: Business dashboard (per-team cost data from admin API, budget utilisation). No queue-depth panel is included in any dashboard. Update docker-compose to mount provisioning directory. Verify dashboards load with docker-compose up. Phase 8 — Admin API + Security Hardening

### ***Prompt 8.1: Admin API***

In `src/gateway/admin/router.py`, add `/admin` routes (require `X-Admin-Key` header):

- `GET /admin/teams` — list teams with current rate-limit status, budget spend, health summary.
- `PUT /admin/teams/{team_id}/limits` — update rpm, tpm, daily/monthly budget in Redis; write audit log to Postgres.
- `POST /admin/teams/{team_id}/keys` — generate new API key: generate  $\geq 128$  bits of secure random material, prepend `llmgw_`, compute `HMAC-SHA256(pepper, full_key)`, store `key_prefix + hmac_digest` in `api_keys` table. Display full key ONCE in response. Never log the key.
- `DELETE /admin/teams/{team_id}/keys/{key_id}` — revoke key (set `revoked_at`).
- `GET /admin/audit-log?team_id=&limit=` — paginated audit log from Postgres. Write integration tests for: update  $\rightarrow$  reflected within 500 ms; key revocation  $\rightarrow$  401 on next use.

### ***Prompt 8.2: Audit log migration (if not in 001\_initial.sql)***

If `audit_log` was not included in `001_initial.sql`, create `database/migrations/002_audit_log.sql`: `CREATE TABLE audit_log ( id BIGSERIAL PRIMARY KEY, actor TEXT NOT NULL, team_id UUID REFERENCES teams(team_id), action TEXT NOT NULL, old_values JSONB, new_values JSONB, timestamp TIMESTAMPTZ NOT NULL DEFAULT now() );` The migration runner introduced in Prompt 2.1 will apply this automatically on startup. Write a test verifying the schema after migration.

Phase 9 — Integration Tests + Load Tests + Docker + CI

### ***Prompt 9.1: Mock provider service for load tests***

Create `tests/load/mock_server.py`: a lightweight FastAPI app implementing:

- POST /v1/chat/completions (OpenAI-compatible)
- POST /v1/messages (Anthropic-compatible) Configurable via env vars: MOCK\_LATENCY\_MS (default 100), MOCK\_FAIL\_RATE (default 0.0). Add GET /mock/stats returning call counts, error counts, and measured\_latency\_ms\_avg. Add to docker-compose as mock-providers service.

### ***Prompt 9.2: Locust load test***

Create tests/load/locustfile.py with:

- InteractiveUser: short prompts (50–100 tokens), ramp to 150 users. Represents interactive chat workload.
- LargeContextUser: long prompts (1,000–4,000 tokens), 50 users. Represents document-heavy workload.
- Custom stats reporter writing CSV: p50\_overhead\_ms, p99\_overhead\_ms, error\_rate, fallback\_rate, budget\_violations. overhead\_ms = total\_latency\_ms - measured\_provider\_call\_duration\_ms (from X-Provider-Duration-Ms response header, set by gateway instrumentation). Run command in README: locust --headless -u 200 -r 10 --run-time 3m --csv results/load\_test Gateway overhead is computed from measured provider-call durations, not from MOCK\_LATENCY\_MS.

### ***Prompt 9.3: Full CI pipeline***

Update .github/workflows/ci.yml with jobs:

- lint: ruff check, ruff format --check, mypy src/
- unit-tests: pytest tests/unit/ --cov=src/gateway --cov-report=xml
- integration-tests: start redis:7-alpine and postgres:15-alpine as services; run pytest tests/integration/
- docker-build: docker build . All jobs run on every PR. No API credentials required.

### ***Prompt 9.4: Demo script***

Create scripts/demo.sh:

- docker-compose up -d
- Wait for /ready endpoint
- Seed 3 demo teams via admin API (create teams, generate API keys)
- Fire 20 requests across tiers
- Simulate primary provider outage (MOCK\_FAIL\_RATE=1.0 for mock-providers)
- Fire 5 more requests; assert fallback activates
- Print summary table: requests sent, fallback count, error count, avg latency, avg overhead
- Print Grafana URL The demo must complete in < 2 minutes.



### ***Prompt 10.1: README***

Write README.md structured as an internal engineering document (not a tutorial). Include:

- **HEADLINE** (first paragraph): one-sentence case study with YOUR ACTUAL load test numbers.
- **ARCHITECTURE**: Mermaid diagram showing all components and data flows, including the `attempt_id` model.
- **ENGINEERING DECISIONS**: For each: (1) distributed token bucket vs leaky bucket, (2) atomic dual-bucket Lua admission, (3) conservative TPM reservation with no post-execution refund vs reconciled TPM accounting — explain the trade-off explicitly (simpler and safer, but underestimates effective throughput for requests that generate far fewer tokens than `max_output`), (4) microdollar integer accounting vs float, (5) HMAC-SHA-256 vs bcrypt for API keys, (6) passive vs active health signals, (7) streaming failure semantics, (8) per-attempt budget reconciliation vs per-request, (9) measured provider-call duration for overhead approximation — describe what it captures and its limitations. Write 2-3 sentences per decision: what, why, what was rejected.
- **QUICK START**: prerequisites → clone → `.env` → docker-compose up → seed → first request → Grafana.
- **LOAD TEST RESULTS**: table with YOUR actual numbers (not PRD targets). Include measured gateway overhead. Tone: internal docs for a new teammate. Never write "this project demonstrates".

### ***Prompt 10.2: CV bullets + interview prep***

Produce two artifacts for CV/interview use:

- **CV BULLET POINTS** (3-4 lines): framed as built X achieving Y with Z metric. Use YOUR actual load test numbers. Strong action verbs.
- **INTERVIEW TALKING POINTS** in STAR format for: "Tell me about a system you built for reliability / fault tolerance."; "How did you handle observability in a distributed system?"; "Walk me through how you approach rate limiting at scale."; "How do you think about cost optimisation with LLM APIs?"; "What trade-offs did you make in the design?"; "How did you test this — how do you know it works under load?" Each answer: 4-6 sentences, one specific technical detail proving depth. Replace placeholder metrics with YOUR actual numbers before using.

## 24. Portfolio Headline

■ Replace X with your ACTUAL load test number after completing Phase 9.

"I built an LLM API gateway handling X req/s with <10 ms overhead, automatic multi-provider failover, and per-team budget enforcement." Every other detail in your CV and README is supporting evidence for this headline.

V2 → V2.1 Revision Notes This section records all changes made from V2.0 to V2.1. No new features are introduced here. All changes address correctness issues identified in V2.0.

### ***Change 1 — Atomic Dual RPM/TPM Admission***

V2.0 used separate Lua operations for RPM and TPM, allowing partial consumption (RPM consumed, TPM rejected, RPM capacity lost). V2.1 implements a single Lua script that evaluates and consumes both buckets atomically. If either bucket lacks capacity, nothing is consumed from either. Tests added proving that a TPM rejection does not consume RPM capacity and vice versa.

### ***Change 2 — TPM Reservation/Reconciliation Semantics***

V2.0 called `check_and_consume(estimated_tokens)` without defining how `estimated_tokens` relates to input tokens, `max_output_tokens`, and actual tokens. V2.1 explicitly defines: before execution, `estimated_tpm_tokens = estimated_input_tokens + max_output_tokens`. After execution, `actual_tpm_tokens = actual_input_tokens + actual_output_tokens`. The design explains why reservation is required, how unused capacity is handled, how streaming is handled, and what happens when authoritative usage is unavailable (conservative policy).

### ***Change 3 — Provider-Attempt Accounting Across Retries and Fallbacks***

V2.0 implied that failed provider attempts always release the entire budget reservation, incorrectly assuming zero billable cost. V2.1 introduces explicit distinction between gateway requests (`request_id`) and provider attempts (`attempt_id`). Each attempt is reserved and reconciled independently. A documented conservative policy applies when authoritative usage is unavailable.

### ***Change 4 — Attempt-Level Idempotency for Budget Reconciliation***

V2.0 used `gateway:reconciled:{request_id}` as the idempotency key, which is insufficient when one request has multiple provider attempts. V2.1 changes the idempotency key to `gateway:reconciled:{request_id}:{attempt_id}`.

Each attempt reconciles exactly once. Tests added proving that reconciling attempt A does not prevent reconciling attempt B, and that duplicate reconciliation of the same attempt is a no-op.

### ***Change 5 — PostgreSQL Phase Dependency Order***

V2.0 introduced PostgreSQL schema in Phase 7 (Admin), but Phase 3 (Auth) already required `api_keys` to exist.

V2.1 introduces a dedicated Phase 2 (PostgreSQL Foundation) that creates the minimum schema (`teams`, `api_keys`) before any auth code is implemented. The `audit_log` table may be added in a later migration. All implementation prompts updated to reflect correct dependency order.

### ***Change 6 — HMAC-SHA-256 API Key Verification***

V2.0 referred to "bcrypt/PBKDF2" interchangeably without committing to one design. V2.1 specifies HMAC-SHA- 256 with a server-side secret pepper as the explicit API key verification method. The rationale (high-entropy machine-generated credentials do not require slow password hashing) is documented. Schema updated to store hmac\_digest (BYTEA) instead of a bcrypt/PBKDF2 hash. Verification algorithm fully specified. Secret pepper sourced from environment, never stored in database or logs.

### ***Change 7 — Measured Gateway Overhead Calculation***

V2.0 computed gateway overhead as total\_latency - MOCK\_LATENCY\_MS, which is incorrect under load when observed mock latency differs from configured latency. V2.1 instruments actual provider-call duration via OTel span timing and computes: gateway\_overhead\_ms = end\_to\_end\_duration - sum(measured\_provider\_call\_durations). Added gateway\_overhead\_ms Prometheus histogram. Locust reporter updated to use per-request measured durations via response header.

### ***Change 8 — Removal of Core Priority-Queue Remnants***

V2.0 had correctly moved priority queues to stretch goals but retained terminology: RealtimeUser (high priority) and BatchUser (low priority) in load tests, and a "queue depth" Grafana panel. V2.1 replaces these with InteractiveUser (short prompts) and LargeContextUser (long prompts) representing traffic patterns, not priorities. Queue depth removed from all core Grafana dashboards. Priority queues remain in the stretch-goals section only.

### ***Change 9 — Neutral Target-Role Positioning***

V2.0 included wording positioning the project specifically as targeting "Senior AI / Platform Engineering Roles" and claiming it reflects skills "mid-level engineers already own". V2.1 uses neutral target-role positioning: Backend Engineer, AI Platform Engineer, AI Infrastructure Engineer, LLM Platform Engineer. The technical depth of the project is unchanged.

#### **Global Consistency Fixes Applied**

- No section describes Redis sorted-set sliding windows as token buckets (token bucket algorithm used throughout).
- No authoritative currency accounting uses Python float.
- No prompt instructs an AI coding assistant to invent current provider pricing.
- RPM + TPM request admission is atomic throughout the document.
- TPM reservation/reconciliation semantics are consistent everywhere.
- Budget reservation and reconciliation work correctly with retries and fallbacks via attempt\_id.
- request\_id and attempt\_id semantics are consistent everywhere.
- Every provider attempt can be independently reconciled exactly once.
- Database schema (teams, api\_keys) exists in Phase 2 before Phase 4 requires it.
- HMAC-SHA-256 terminology is consistent; bcrypt/PBKDF2 references removed.
- No API key, secret pepper, prompt, completion, or Authorization header is logged.

- Gateway overhead uses measured provider-call duration rather than configured mock latency.
- Priority queues remain stretch goals only; no queue-depth metric in core dashboards.
- Mock providers remain the default for all automated tests.
- Redis concurrency/Lua tests use real Redis.
- No real paid provider API call is required by CI.
- Streaming failure semantics unchanged: fallback allowed before first byte; no transparent fallback after streaming begins.
- Prometheus labels remain low-cardinality.
- README/CV metrics must come from actual measured load-test results.
- Health/readiness and graceful startup/shutdown behaviour remain intact.
- Retry, circuit breaker, and fallback behaviour are internally consistent with attempt\_id model.
- Every implementation prompt is ordered so it only relies on files, schemas, classes, and infrastructure created in earlier prompts.

V2.1 → V2.2 Revision Notes This section records all changes made from V2.1 to V2.2. No new features are introduced. All changes address residual ambiguities identified in V2.1.

### ***Correction 1 — TPM Rate-Limit Reservation Semantics (Section 8.3, 7.7, 18.2, 18.3, Prompts***

4.3, 10.1) V2.1 Section 8.3 created ambiguity between two separate systems: (a) TPM token-bucket rate-limit accounting in Redis, and (b) financial budget reservation/reconciliation in microdollars. Specifically, V2.1 stated that "unused reserved TPM capacity is effectively released" and that "reconciliation operates on budget microdollars which naturally reflects the actual token cost" — wording that incorrectly implied TPM tokens are returned after execution.

V2.2 correction:

- Section 8.3 is rewritten to explicitly separate the two systems. TPM rate-limit reservation is conservative and final: `estimated_tpm_tokens = estimated_input_tokens + max_output_tokens` is charged at admission and is NOT refunded after execution. This is documented as an intentional engineering trade-off.
- Section 8.4 is renamed "Financial Budget Enforcement" to make clear it covers microdollar accounting only, and does not touch the TPM bucket during reconciliation.
- Section 7.7 adds an acceptance criterion: the TPM bucket must remain charged for the original estimated amount after completion with fewer actual tokens.
- Section 18.2 adds `test_conservative_tpm_no_refund` to Redis integration tests.
- Section 18.3 adds `test_tpm_conservative_no_refund` to the full integration test table.
- Prompt 4.3 adds `test_conservative_tpm_no_refund` and makes explicit that the Lua script has no refund path.
- Prompt 10.1 (README) adds conservative TPM reservation as a required engineering decision with trade-off explanation.
- Tests do NOT expect TPM capacity to be restored after request completion.

### ***Correction 2 — Gateway Overhead Measurement Clarification (Section 14.5, Prompt 9.2,***

Prompt 7.3) V2.1 Section 14.5 contained a contradictory statement: it said the metric "includes network round-trip time to the provider and cannot isolate pure gateway CPU cost." However, the `measured_provider_call_duration_ms` span wraps the full provider adapter call including the network round-trip — so the network IS subtracted out, not included in the overhead figure. The statement was self-contradictory.

V2.2 correction:

- Section 14.5 removes the contradictory sentence. The metric is now described as an application-level approximation, not a pure measurement.

- The overhead formula is restated clearly:  $\text{gateway\_overhead\_ms} = \text{total\_request\_duration\_ms} - \text{sum}(\text{measured\_provider\_call\_duration\_ms})$ . Because the provider network round-trip is inside the measured span, it is subtracted out.
- The section now explicitly lists what the metric may still include despite subtracting provider-call duration: framework scheduling, event-loop scheduling, instrumentation overhead, serialisation/deserialisation outside the provider timing boundary, and measurement error under concurrency.
- Prompt 9.2 (Locust) already correctly uses measured per-request provider duration from the response header — this is confirmed consistent.
- Prompt 7.3 already correctly describes `gateway_overhead_ms` as computed from measured durations — confirmed consistent.
- The benchmark report requirement is updated to state that the timing boundary must be clearly documented. Blocking Technical Inconsistencies After V2.2 None. No blocking technical inconsistency remains. The following items were verified consistent and required no change:
  - `request_id` and `attempt_id` semantics are consistent throughout.
  - Budget reconciliation remains at the provider-attempt level.
  - HMAC-SHA-256 with server-side pepper is the only API-key verification design.
  - PostgreSQL minimum schema (`teams`, `api_keys`) is established in Phase 2 before Phase 4 (auth) requires it.
  - Atomic RPM+TPM admission remains one Lua operation.
  - Priority queues remain a stretch goal only; no queue-depth metric exists in core dashboards.
  - `InteractiveUser` and `LargeContextUser` are traffic patterns, not priorities.
  - README/CV metrics must come from actual measured load-test results.
  - No implementation prompt contradicts an earlier phase.
  - No test expects functionality not yet implemented at that phase. Optional Improvements (Not Applied — Specification Frozen) The following optional improvements were identified during the consistency review. They do not constitute blocking inconsistencies and the PRD is not modified for them:
    - OPTIONAL: A future revision could add an explicit note in `TeamConfig` that `rate_limit_tpm` should be sized accounting for the conservative TPM admission policy (i.e. `teams` should set `limit_tpm` based on estimated token usage including `max_output_tokens`, not actual token usage). This is a usage guidance note, not an architectural requirement.
    - OPTIONAL: The benchmark report template in Prompt 10.1 could include a suggested table row for "effective TPM throughput vs configured TPM limit" to help demonstrate the trade-off in practice. Not required for correctness.

- OPTIONAL: A comment in pricing.yaml could note that `cost_estimate_microdollars` uses `max_output_tokens` (conservative), while actual billed cost uses `actual_output_tokens`. Cosmetic only. These optional items may be addressed during implementation if desired. They do not affect the correctness of the specification.